

Computer Vision Cheat Sheet — 02504

Based on DTU Course 02504 exercises and lecture notes by Henrik Aanæs

1. Homogeneous Coordinates

A 2D point (x, y) in homogeneous coordinates:

$$\tilde{q} = \begin{bmatrix} sx \\ sy \\ s \end{bmatrix} \quad (\text{any } s \neq 0)$$

A 3D point (X, Y, Z) in homogeneous coordinates:

$$\tilde{Q} = \begin{bmatrix} sX \\ sY \\ sZ \\ s \end{bmatrix}$$

Converting back (inhomogeneous): divide all elements by the last coordinate.

Line in 2D: $l = [a, b, c]^T$ such that $l^T q = 0$, i.e. $ax + by + c = 0$

Intersection of two lines: $q = l_1 \times l_2$

Line through two points: $l = q_1 \times q_2$

Distance from point to line (where $a^2 + b^2 = 1$):

$$d = l^T \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = ax + by + c$$

Key insight — scale invariance: In homogeneous coordinates, q and sq (for any scalar $s \neq 0$) represent the *same point*. This is why many CV matrices are “defined up to scale” — multiplying by a scalar does not change what they represent, because the homogeneous division cancels it out.

2. Scale Invariance of CV Matrices

A matrix is **NOT** scale-equivalent if multiplying by an arbitrary scalar changes its meaning.

Matrix	Scale equivalent?	Reason
Homography H	Yes	Used in homogeneous coords
Projection P	Yes	sPQ is same point as PQ
Fundamental F	Yes	$q^T(sF)q = 0$ iff $q^T F q = 0$
Essential E	Yes	Same epipolar geometry
Camera K	Yes	Used in homogeneous projection
Rotation R	NO	Must satisfy $R^T R = I$, $\det(R) = 1$ $7R$ violates these hard constraints

Only the rotation matrix R is NOT scale-equivalent. All other matrices in the table operate in homogeneous coordinates where scale cancels out. R is special because it has absolute geometric constraints that scaling destroys.

3. Degrees of Freedom and Minimum Correspondences

Each point correspondence gives a fixed number of equations. You need at least as many equations as unknowns (DOF).

Matrix	DOF	Min. points	Reason
Homography H	8	4	$9 - 1$ (scale); each pt gives 2 eqs
Fundamental F	7	7 or 8	$9 - 1$ (scale) -1 (rank)
Essential E	5	5	$= 3$ (rotation) $+2$ (translation dir.)
Projection P	11	6	$12 - 1$ (scale); each pt gives 2 eqs

Why E has 5 DOF (not 7 like F):

- E encodes the relative pose of two **calibrated** cameras
- Rotation R : 3 DOF
- Translation direction t (scale unknown, so only direction): 2 DOF
- Total: **5 DOF**
- Extra constraint vs F : singular values of E must be $\{s, s, 0\}$ — both non-zero values must be **equal**

Why H needs 4 points (not 3):

Each 2D–2D correspondence gives **2 equations**. With 4 points: $4 \times 2 = 8$ equations for 8 unknowns. With 3 points: only 6 equations — underdetermined.

4. Pinhole Camera Model

$$q_i = PQ_i, \quad P = K \begin{bmatrix} R & t \end{bmatrix}$$

Intrinsic matrix K :

$$K = A = \begin{bmatrix} f & f\beta & \Delta x \\ 0 & \alpha f & \Delta y \\ 0 & 0 & 1 \end{bmatrix}$$

- f = focal length (in pixels)
- $\Delta x, \Delta y$ = principal point (usually $\approx$ image centre)
- $\alpha \approx 1$ (pixel aspect ratio), $\beta \approx 0$ (skew)

Inhomogeneous projection:

$$x_i = \frac{p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14}}{p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}}, \quad y_i = \frac{p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24}}{p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}}$$

Camera centre (world coordinates):

$$\tilde{Q}_c = -R^T t$$

Important: Camera position $\neq$ translation vector t !

Field of view:

$$\theta = 2 \arctan\left(\frac{l/2}{f}\right)$$

5. Radial Distortion

$$p_i^d = A_p \begin{bmatrix} R & t \end{bmatrix} Q_i$$

$$\begin{bmatrix} x_i^c \\ y_i^c \end{bmatrix} = \begin{bmatrix} x_i^d \\ y_i^d \end{bmatrix} (1 + \Delta(r_i)), \quad q_i = A_q p_i^c$$

$$\Delta(r) = k_3 r^2 + k_5 r^4 + k_7 r^6 + \dots, \quad r_i = \sqrt{(x^d)^2 + (y^d)^2}$$

$$A_p = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A_q = \begin{bmatrix} 1 & \beta & \Delta x \\ 0 & \alpha & \Delta y \\ 0 & 0 & 1 \end{bmatrix}$$

6. Coordinate Transforms

Change of frame: $Q' = RQ + t$

Inverse: $R' = R^T$, $t' = -R^T t$

Note: $R^{-1} = R^T$ for rotation matrices.

Relative rotation and translation (used in visual odometry):

$$R_{rel} = R_2 R_1^T, \quad t_{rel} = t_2 - R_2 R_1^T t_1$$

7. Direct Linear Transform (DLT)

For homography

— each pair $(x, y) \leftrightarrow (u, v)$ gives 2 rows:

$$A_i = \begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & ux & uy & u \\ 0 & 0 & 0 & -x & -y & -1 & vx & vy & v \end{bmatrix}$$

For projection matrix

— each $(X, Y, Z) \leftrightarrow (u, v)$ gives 2 rows:

$$A_i = \begin{bmatrix} X & Y & Z & 1 & 0 & 0 & 0 & 0 & -uX & -uY \\ -uZ & -u & & & X & Y & Z & 1 & -vX & -vY \\ 0 & 0 & 0 & 0 & & & & & & \\ -vZ & -v & & & & & & & & \end{bmatrix}$$

Solution: Stack all rows into matrix A , solve $Ah = 0$ via SVD. Solution = last row of V^T .

Normalisation: Centre points to mean 0, scale to std 1. Denormalise after:

$$H = T_2^{-1} H_{\text{norm}} T_1$$

Normalisation improves numerical precision from $\sim 10^{-11}$ to $\sim 10^{-14}$.

Key Matrix Glossary — What Each Matrix Is and When to Use It

Matrix	What it is	When to use it
K (Camera / Intrinsic)	Encodes the physical properties of a single camera: focal length f , principal point $(\Delta x, \Delta y)$, skew β	Always needed for projection. Found via calibration (Zhang's method).
R (Rotation)	3×3 orthogonal matrix describing camera orientation in world. Must satisfy $R^T R = I$, $\det(R) = 1$	Part of every projection matrix $P = K[R t]$. Recovered from E in visual odometry.
t (Translation)	3×1 vector describing camera position in its own coordinate frame. Camera world position $= -R^T t$	Part of every projection P . Only direction known from images (scale ambiguous).
P (Projection)	3×4 matrix mapping 3D world points to 2D pixels. $P = K[R t]$	Use for projecting 3D points to image, triangulation, DLT estimation.
H (Homography)	3×3 matrix mapping 2D pixels in one image to 2D pixels in another. 8 DOF.	Use when scene is flat (checkerboard) or camera only rotates ($t = 0$). Image stitching, Zhang's calibration.
F (Fundamental)	3×3 rank-2 matrix encoding epipolar geometry between two uncalibrated cameras. 7 DOF. Works in pixel coordinates.	Use when K is unknown . Gives epipolar lines: $l_2 = F q_1$, $l_1 = F^T q_2$.
E (Essential)	3×3 matrix encoding epipolar geometry between two calibrated cameras. 5 DOF. Works in normalised coordinates.	Use when K is known . Can decompose into R and t . Used in visual odometry.
$B = K^{-T} K^{-1}$	Symmetric 3×3 matrix encoding K in a form that can be solved linearly. Packed into vector b .	Used internally in Zhang's calibration to recover K from homographies via $Vb = 0$.

E vs F — the key distinction:

Both describe how a point in one image constrains where it appears in the other (epipolar line). The difference is whether K is known:

- **Don't know** $K \rightarrow$ estimate F (8-point algorithm, pixel coordinates)
- **Know** $K \rightarrow$ estimate E (5-point algorithm, normalised coords), then recover R, t

Relationship: $E = K_2^T F K_1$ and $F = K_2^{-T} E K_1^{-1}$

8. Epipolar Geometry

Setup: $P_1 = K_1[I|0]$, $P_2 = K_2[R|t]$

Essential matrix: $E = [t]_{\times} R$

Fundamental matrix:

$$F = K_2^{-T} [t]_{\times} R K_1^{-1} = K_2^{-T} E K_1^{-1}$$

Epipolar constraint: $q_2^T F q_1 = 0$

Epipolar line in image 2 from point in image 1: $l_2 = F q_1$

Epipolar line in image 1 from point in image 2: $l_1 = F^T q_2$

Properties:

- F : 7 DOF, rank 2, $\det(F) = 0$
- E : 5 DOF, singular values $\{s, s, 0\}$ (both non-zero equal)
- E is a special case of F for calibrated cameras ($K_1 = K_2 = I$)

Cross-product operator (skew-symmetric):

$$[t]_{\times} = \begin{bmatrix} 0 & -t_3 & t_2 \\ t_3 & 0 & -t_1 \\ -t_2 & t_1 & 0 \end{bmatrix}$$

9. Homography

A 3×3 matrix H mapping 2D homogeneous coordinates: $q_1 = H q_2$

- 8 DOF; requires minimum **4 point correspondences**
- Use when scene is flat or $t = 0$ (pure rotation)
- F cannot be estimated from a purely planar scene — use H instead

Transfer error (for RANSAC inlier check):

$$d = \|\tilde{q}_2 - \Pi(H q_1)\|_2$$

10. Triangulation

Linear (DLT):

Each camera gives 2 rows:

$$A_i = \begin{bmatrix} u_i P_i^{3T} - P_i^{1T} \\ v_i P_i^{3T} - P_i^{2T} \end{bmatrix}$$

Solve $AQ = 0$ via SVD.

Reprojection RMSE:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i \|\Pi(P_i Q) - \tilde{q}_i\|_2^2}$$

Nonlinear triangulation (Levenberg-Marquardt):

Correct function for LM triangulation:

Input: inhomogeneous 3D point $\in \mathbb{R}^3$ (NOT $\mathbb{R}^4$)

Why not $\mathbb{R}^4$: homogeneous coords have arbitrary scale — the optimiser cannot distinguish real progress from rescaling.

Output: vector of residuals $\in \mathbb{R}^{n \cdot 2}$ (2 residuals per camera, not scalar distances)

Why residuals not distances: LM minimises $\|f(x)\|_2^2$ — it needs a *vector* of differences.

For $n = 3$ cameras: $f : \mathbb{R}^3 \rightarrow \mathbb{R}^6$

$$\min_{Q \in \mathbb{R}^3} \sum_i \|\Pi(P_i Q) - \tilde{q}_i\|_2^2$$

Use linear triangulation as initial guess x_0 , then `scipy.optimize.least_squares`.

11. Zhang's Camera Calibration

Key facts about Zhang's algorithm:

- Requires **flat** calibration objects (checkerboard, $z = 0$ plane)
- Uses a series of **homographies** — one per view
- Each homography imposes linear constraints on b (encoding K), NOT on F
- Solved via **SVD**, not direct decomposition of P
- Works for any field of view; minimum **3 views** needed
- Does NOT adjust the physical lens

What R and t describe per calibration view:

For the i 'th calibration object, R and t describe the coordinate transformation **from the i 'th calibration object to the camera**.

The formula $q = K[R|t]Q$ takes Q expressed in checkerboard coordinates and transforms it to camera coordinates before projection. The camera is always the reference frame.

B matrix:

$B = K^{-T}K^{-1}$, packed as $b = [B_{11}, B_{12}, B_{22}, B_{13}, B_{23}, B_{33}]^T$

Constraint per view:

$Vb = 0$, where:

$$V = \begin{bmatrix} v_{12}^T \\ (v_{11} - v_{22})^T \end{bmatrix}, \quad v_{ij} = \begin{bmatrix} h_{i1}h_{j1} \\ h_{i1}h_{j2} + h_{i2}h_{j1} \\ h_{i2}h_{j2} \\ h_{i3}h_{j1} + h_{i1}h_{j3} \\ h_{i3}h_{j2} + h_{i2}h_{j3} \\ h_{i3}h_{j3} \end{bmatrix}$$

Extrinsics from H :

$$\lambda = \frac{1}{\|K^{-1}h_1\|}, \quad r_1 = \lambda K^{-1}h_1, \quad r_2 = \lambda K^{-1}h_2, \quad t = \lambda K^{-1}h_3, \quad r_3 = r_1 \times r_2$$

12. Harris Corner Detector

Step-by-step computation (useful for exam):

1. Read $a = g * (I_x^2)$, $b = g * (I_y^2)$, $c = g * (I_x I_y)$ at pixel (row, col) from the three tables
2. Compute Harris measure: $r = ab - c^2 - k(a+b)^2$, $k = 0.06$
3. Check if $r > \tau$ (threshold)
4. Apply non-maximum suppression: check r is larger than all 4 direct neighbours

Structure tensor:

$$C(x, y) = \begin{bmatrix} g_\varepsilon * I_x^2 & g_\varepsilon * I_x I_y \\ g_\varepsilon * I_x I_y & g_\varepsilon * I_y^2 \end{bmatrix} = \begin{bmatrix} a & c \\ c & b \end{bmatrix}$$

Interpretation of r :

- $r > 0$ large $\rightarrow$ corner (both eigenvalues large)
- $r < 0 \rightarrow$ edge (one eigenvalue much larger than the other)
- $r \approx 0 \rightarrow$ flat region (both eigenvalues small)

Why non-maximum suppression is needed:

Harris gives a *blob* of high r values around each corner, not a single point. Without NMS you'd

detect 5–10 pixels all claiming to be the same corner. NMS keeps only the peak — the pixel higher than all 4 direct neighbours (up, down, left, right). Result: exactly 1 clean detection per corner.

13. Gaussian Smoothing & Derivatives

$$g(x) = e^{-\frac{x^2}{2\sigma^2}}, \quad g_d(x) = -\frac{x}{\sigma^2}g(x)$$

Kernel length: $\pm 3\sigma$ captures 99.99% of distribution.

Truncation error:

- $\pm 1\sigma$: $\sim 32\%$ of area missing (too short)
- $\pm 2\sigma$: $\sim 5\%$ missing (acceptable)
- $\pm 3\sigma$: $\sim 0.01\%$ missing (standard choice)

Separable 2D smoothing:

$$I = g *_x (g *_y \text{im}), \quad I_x = g_d *_x (g *_y \text{im}), \quad I_y = g *_x (g_d *_y \text{im})$$

14. Blob Detection (DoG)

Scale space: $\sigma_i = \sigma \cdot 2^i$, for $i = 0, \dots, n-1$

$$\text{DoG}_i = \text{im}_{i+1} - \text{im}_i$$

Blob detection conditions:

- $|\text{DoG}(x, y, i)| > \tau$ (threshold)
- Larger than 8 spatial neighbours at same scale
- Larger than max-filtered DoG at scale above and below

Circle radius for visualisation $\propto \sigma \cdot 2^i$.

15. RANSAC

Algorithm:

1. Sample **minimum** points needed to fit the model
2. Fit model to those points
3. Count inliers: all points with distance $< \varepsilon$ to the model
4. Keep best model so far; repeat N iterations
5. Refit to all inliers of best model (e.g. PCA for lines)

Inlier threshold:

$$\varepsilon = 3.84 \cdot \sigma^2$$

where 3.84 is the χ^2 value at 95% confidence (1 DOF) and σ is the assumed pixel noise std.

Adaptive stopping criteria:

$$N = \frac{\log(1-p)}{\log(1-(1-\hat{\varepsilon})^n)}, \quad \hat{\varepsilon} = 1 - \frac{s}{m}$$

- p = desired probability of success (e.g. 0.99 or 0.95)
- $\hat{\varepsilon}$ = estimated outlier ratio
- s = current best inlier count, m = total number of points
- n = **sample size** — depends on the model!

Sample size n per model — common exam trap:

Model	n
Line fitting	$n = 2$
Homography H	$n = 4$
Fundamental matrix F	$n = 8$ (linear) or $n = 7$
Essential matrix E	$n = 5$

Common mistake: using $n = 2$ (line) when estimating a homography ($n = 4$). Always identify which model you are fitting first!

Sampson distance (for fundamental matrix):

$$d_S = \frac{(q_2^T F q_1)^2}{(F q_1)_1^2 + (F q_1)_2^2 + (F^T q_2)_1^2 + (F^T q_2)_2^2}$$

16. SIFT Features

1. Build DoG scale space $\rightarrow$ find keypoints at extrema across scale and space
2. Assign dominant orientation from local gradient histogram
3. Compute 128-dimensional descriptor from 4×4 grid of gradient histograms

Invariant to: scale, rotation, illumination changes. Partially invariant to viewpoint.

Matching strategies:

- **Cross-checking:** $A \rightarrow B$ and $B \rightarrow A$ must agree. Stricter, fewer but cleaner matches.
- **Ratio test (Lowe):** keep match if $d_1/d_2 < 0.75$. Removes ambiguous matches.

SIFT vs Structured Light:

- SIFT: sparse (~ 2000 keypoints), works with ordinary images, no special lighting needed
- Structured light: dense ($\sim 100k+$ pixels matched), requires projector, much more precise

17. Image Stitching (Panorama)

1. Detect SIFT keypoints and descriptors in both images
2. Match with cross-checking
3. RANSAC to estimate homography H (use $n = 4$, threshold $\tau \approx 10\text{--}20$ px)
4. Warp one image using H
5. Composite images using masks

Automatic range: Warp the 4 corners of each image through H ; take the bounding box of all warped corners to set xRange and yRange automatically — no manual tuning needed.

18. 8-Point Algorithm (Fundamental Matrix)

Each point correspondence $(x_1, y_1) \leftrightarrow (x_2, y_2)$ gives one row:

$$A_i = [x_2 x_1, x_2 y_1, x_2, y_2 x_1, y_2 y_1, y_2, x_1, y_1, 1]$$

Solve $Ah = 0$ via SVD. **Enforce rank-2** by zeroing the smallest singular value of the reshaped F . Denormalise:

$$F = T_2^T F_{\text{norm}} T_1$$

19. Visual Odometry

Pipeline for each new frame:

1. SIFT match consecutive frames
2. Estimate E via RANSAC (`cv2.findEssentialMat`)
3. Recover pose R, t (`cv2.recoverPose`) — only *direction* of t known, not scale!

4. Track features through 3 consecutive frames
5. Triangulate 3D points from frames i and $i + 1$
6. Estimate pose of frame $i + 2$ using PnP (`cv2.solvePnP`)

Camera position (NOT the translation vector):

$$\text{pos} = -R^T t$$

Scale ambiguity: translation t only gives direction, not metric distance. External reference needed (GPS, known object size) for true metric scale.

20. Structured Light (3D Scanning)

Pipeline:

1. Project sinusoidal patterns onto the object
2. Capture with two calibrated cameras
3. Decode phase at each pixel using FFT
4. Rectify images so epipolar lines are horizontal rows
5. Match pixels by phase similarity on the same row only (1D search)
6. Triangulate matched pixels $\rightarrow$ dense 3D point cloud

Phase unwrapping (heterodyne principle):

$$\theta_{\text{prim}} = \angle(\text{FFT}(\text{primary images})[1]), \quad N_1 = 40 \text{ periods}$$

$$\theta_{\text{sec}} = \angle(\text{FFT}(\text{secondary images})[1]), \quad N_2 = 41 \text{ periods}$$

$$\theta_c = \angle\left(e^{j(\theta_{\text{prim}} - \theta_{\text{sec}})}\right) \quad (\text{phase cue})$$

$$o = \text{round}\left(\frac{N_1 \theta_c - \theta_{\text{prim}}}{2\pi}\right) \quad (\text{order / fringe number})$$

$$\theta = \frac{\theta_{\text{prim}} + 2\pi o}{N_1} \quad (\text{final unwrapped phase})$$

Mask: Valid pixels where (projector ON image) – (projector OFF image) > threshold (≈ 15).

Rectification: Warp both images so epipolar lines become horizontal. Matching then only searches along each row instead of the full 2D image.

21. Key OpenCV / NumPy Functions

Task	Function
SVD	<code>U, S, Vt = np.linalg.svd(A)</code>
Solution to $Ah = 0$	<code>Vt[-1].reshape(...)</code>
Cross product	<code>np.cross(a, b)</code>
Camera calibration	<code>cv2.calibrateCamera(...)</code>
Stereo rectify	<code>cv2.stereoRectify(...)</code>
Find essential matrix	<code>cv2.findEssentialMat(p1, p2, K)</code>
Recover pose	<code>cv2.recoverPose(E, p1, p2, K)</code>
Triangulate	<code>cv2.triangulatePoints(P1, P2, q1, q2)</code>
PnP RANSAC	<code>cv2.solvePnP(Ransac(Q, q, K, dist)</code>
SIFT	<code>cv2.SIFT_create(nfeatures=2000)</code>
Find homography	<code>cv2.findHomography(src, dst, cv2.RANSAC)</code>
Warp image	<code>cv2.warpPerspective(im, H, outSize)</code>
Canny edges	<code>cv2.Canny(im, t1, t2)</code>
Hough transform	<code>skimage.transform.hough_line(edges)</code>
Rodrigues (rvec $\rightarrow R$)	<code>R, _ = cv2.Rodrigues(rvec)</code>
Nonlinear optimisation	<code>scipy.optimize.least_squares(f, x0)</code>

22. Summary of Key Relationships

Projection pipeline:

$$Q_{3D} \xrightarrow{P=K[R|t]} \tilde{q}_{\text{homogeneous}} \xrightarrow{\Pi} (x, y)_{\text{pixels}}$$

Epipolar geometry:

$$q_2^T F q_1 = 0, \quad F = K_2^{-T} E K_1^{-1}, \quad E = [t]_{\times} R$$

Zhang calibration chain:

$$\text{Checkerboard corners} \rightarrow H_i \rightarrow Vb = 0 \rightarrow b \rightarrow K \rightarrow R_i, t_i \text{ per view}$$

Visual odometry chain:

$$\text{SIFT matches} \rightarrow E \xrightarrow{\text{recoverPose}} R, t \rightarrow \text{Triangulate 3D} \xrightarrow{\text{PnP}} \text{next } R, t$$

Structured light chain:

$$\text{Phase patterns} \rightarrow \text{FFT} \rightarrow \theta \rightarrow \text{Phase match (same row)} \rightarrow \text{Triangulate} \rightarrow \text{3D cloud}$$